

Demo: Mobile RouteJump: An Efficient Mobile Agent with Vision–XML Routing and Macro Actions

Wenrui Lu

The Chinese University of Hong Kong
Hong Kong, Hong Kong SAR, China

Mu Yuan

The Chinese University of Hong Kong
Hong Kong, Hong Kong SAR, China

Bufang Yang

The Chinese University of Hong Kong
Hong Kong, Hong Kong SAR, China

Kaiwei Liu

The Chinese University of Hong Kong
Hong Kong, Hong Kong SAR, China

Muye Jia

The Chinese University of Hong Kong
Hong Kong, Hong Kong SAR, China

Wei Gao

University of Pittsburgh
Pittsburgh, Pennsylvania, USA

Zhenyu Yan

The Chinese University of Hong Kong
Hong Kong, Hong Kong SAR, China

Guoliang Xing

The Chinese University of Hong Kong
Hong Kong, Hong Kong SAR, China

CCS Concepts

• Human-centered computing → Ubiquitous and mobile computing; • Computing methodologies → Artificial intelligence.

Keywords

Mobile Agents, Mobile Task Automation, Large language Models

ACM Reference Format:

Wenrui Lu, Mu Yuan, Bufang Yang, Kaiwei Liu, Muye Jia, Wei Gao, Zhenyu Yan, and Guoliang Xing. 2026. Demo: Mobile RouteJump: An Efficient Mobile Agent with Vision–XML Routing and Macro Actions. In *The 24th Annual International Conference on Mobile Systems, Applications and Services (MobiSys Companion '26)*, June 21–25, 2026, Cambridge, United Kingdom. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3812835.3814965>

1 Introduction

Recent advances in Large Language Models (LLMs) have enabled GUI (graphical user interface) mobile agents that can automate user tasks through natural language interaction with app interfaces. Unlike backend API agents or text-only assistants, GUI agents must repeatedly observe the current screen, identify the correct interface target, and execute low-level GUI actions such as tapping, scrolling, and typing [5]. We develop Mobile RouteJump, an efficient LLM-based mobile GUI agent that improves both UI perception and task execution on real smartphones. Mobile RouteJump addresses two key efficiency bottlenecks in existing mobile agents.

Efficiency bottleneck–1: Perception. GUI agents typically rely on vision (screenshot) and structured XML representations of the current screen to perceive the UI state. *Vision-based agents* [4] feed screenshots into a vision-language model (VLM) to “see” the interface and leverage visual cues like icons, images, and layouts. However, processing a single screenshot with VLMs for each screen is slow and expensive, making vision-based UI understanding unsuitable for multi-screen interactions. To reduce compute overhead,

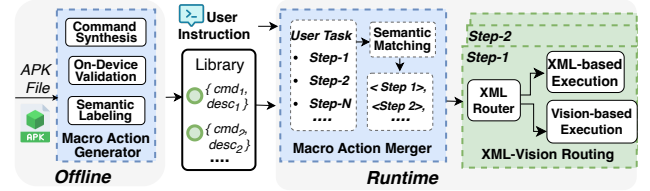


Figure 1: System overview of Mobile RouteJump.

some agents perceive the screen by using *text-based XML representations* which include element types, text content, content descriptions, and layout metadata [5]. Although efficient, XML representations lack pixel-level information that leads to poor performance in UI interpreting, such as identifying unlabeled image buttons and understanding visual thumbnails.

Efficiency bottleneck–2: Execution. After perception, during the planning and executing stage, current agents suffer significant inefficiency because they execute tasks as long sequences of basic UI actions (taps, swipes, typing), invoking the LLM at nearly every step [4, 5]. While flexible, this fine-grained planning wastes computation on routine sub-tasks that follow predictable patterns. For example, searching within an app almost always involves tapping the search box, entering a query, and pressing the submit button—steps that are deterministic and do not require fresh reasoning. We find that many multi-step traces across apps share highly similar action patterns, differing only in parameters (e.g., the text entered).

2 Mobile RouteJump Design

To tackle these challenges, we propose Mobile RouteJump, a LLM-based mobile GUI agent framework that achieves both perception and execution efficiency through two key innovations: *adaptive modality routing* and *dynamic macro actions optimization*. Figure 1 presents the end-to-end architecture of Mobile RouteJump, which operates through an offline profiling phase followed by a runtime execution phase.

During offline profiling, the system performs **Intent–Command Synthesis**, where it parses each target application’s APK to enumerate launchable activities and extract all intent-based entry points.



This work is licensed under a Creative Commons Attribution 4.0 International License. *MobiSys Companion '26*, Cambridge, United Kingdom
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2711-5/2026/08
<https://doi.org/10.1145/3812835.3814965>

It then instantiates these entry points as executable candidate commands. Next, it performs **② On-Device Validation & Semantic Labeling**, in which the system executes each command on real devices, filters out unstable transitions, and annotates every reliable “jump-to-screen” behavior with a natural-language description. Together, these two stages produce a per-app library of validated macro actions, where each macro is represented by an executable command and its screen-level semantics.

During the runtime execution phase, given a natural-language user instruction, Mobile RouteJump first produces a basic action plan consisting of fine-grained micro UI steps such as taps, swipes, and text inputs¹. The system then applies the Macro Action Merger to integrate offline-validated macro actions into this plan. It begins with **③ Macro Semantic Matching**, which scans the step-wise micro plan and identifies subsequences whose semantics align with those of known macro actions in the library. Once a suitable correspondence is identified, **④ Plan Rewriting** replaces the matched subsequence with the higher-level macro, thereby producing a macro-augmented execution plan that preserves the original task intent while reducing plan length and minimizing LLM invocations.

Finally, for each step of the macro-augmented plan, the Mobile RouteJump invokes its **⑤ Router Agent**, which first attempts XML-based execution by filtering the current UI hierarchy into a concise set of actionable elements and aligning the required operation with these structural cues. When the XML information is insufficient—such as when UI elements contain ambiguous text or the target is purely visual—the router escalates to screenshot-based visual analysis. By prioritizing XML and invoking vision only when necessary, Mobile RouteJump achieves latency and token efficiency while preserving robust perception capabilities across visually complex interfaces.

3 Evaluation Summary

We test Mobile RouteJump across 124 real-world tasks on 6 popular Android apps, spanning a self-collected RouteJump Benchmark and public Baseline Dataset Combined. Table 1 show the overall and ablation performance of Mobile RouteJump. Results show that overall Mobile RouteJump achieves up to 91.4% task success rate and up to 3.9× speedup over strong vision-only MobileAgent-V2 and XML-only MobileGPT baselines [2, 4]. Ablation results further show that the validated macro actions reduce up to 33.3% latency and 25.0% token cost. Our microbenchmarks also shows that adaptive vision–XML routing avoids up to 85.2% of unnecessary screenshot-based reasoning. It also confirm that visual operations, though used for only a small fraction of steps, incur roughly 5× higher per-step latency and 10–15× higher token cost than XML-based actions. For all inference modules and all baselines, we use GPT-4o [1] as the LLM backend via the Poe API². Further details of the experimental setup, microbenchmarks, and complete results can be found in the conference version of this paper.

4 Demo Description

Our demo consists of a OnePlus Ace 3 Pro smartphone running the target applications, a laptop hosting the Mobile RouteJump backend

¹This step can be implemented by existing tools such as Seq2Act [3] and hence is not included in the design of Mobile RouteJump.

²<https://poe.com>

Overview								
Methods	RouteJump Bench			Baseline		Combined		
	Success	Latency		Success		Latency		
MobileGPT	83.5%	79.7s		79.23%		52.3s		
MobileAgent-V2	87.2%	123.9s		82.0%		94.1s		
Mobile RouteJump	91.4%	31.6s		87.3%		26.1s		

Ablation								
Mer-ger	Ablation Setting		Mobile RouteJump Bench			Baseline Combined		
	XML Execution	Visual Execution	SR	LA	CO	SR	LA	CO
-	+	+	92.9	47.4	0.18	86.3	35.7	0.16
+	+	-	86.0	20.4	0.05	80.8	15.2	0.05
+	-	+	88.3	91.0	0.58	83.5	72.1	0.43
+	+	+	91.3	31.6	0.14	87.3	26.1	0.12

Table 1: Overall performance (top): task success rate and latency per task across datasets. Ablation (bottom): macro-action merger, XML execution, and visual execution. SR: success rate (%), LA: latency (s), CO: cost in US dollars.

and visualization dashboard. During the demonstration, attendees observe both the real GUI execution on the phone and the internal decision process of the agent. The dashboard presents the signals that make these mechanisms visible to the audience. Specifically, it shows the live phone screen, the current user instruction, a filtered XML candidate view, the router’s decision on whether the step is handled by XML or escalated to vision, and the matched macro action whenever a validated shortcut is used. In addition, the dashboard displays lightweight runtime statistics such as action count, latency, and token-cost, allowing attendees to directly observe how XML-first execution and macro-based plan rewriting reduce unnecessary visual calls and shorten routine interaction traces.

We demonstrate Mobile RouteJump using a set of representative tasks chosen to expose its core behaviors. For example, an app search-and-play task illustrates mixed-modality execution, an app settings task, such as enabling the break-reminder option and setting the interval, illustrates macro-based execution, etc. The significance of this demo lies in making Mobile RouteJump’s efficiency mechanisms visible during live execution. Attendees can directly observe how adaptive Vision–XML routing reduces unnecessary visual reasoning and how validated macro actions shorten routine GUI interaction traces, providing a concrete view of the efficiency of Mobile RouteJump.

Acknowledgments

This work is supported by Research Grants Council of Hong Kong under grants T43-513/23-N.

References

- [1] Aaron Hurst et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).
- [2] Sunjae Lee et al. 2024. Mobilegpt: Augmenting llm with human-like app memory for mobile task automation. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 1119–1133.
- [3] Yang Li et al. 2020. Mapping natural language instructions to mobile UI action sequences. *arXiv preprint arXiv:2005.03776* (2020).
- [4] Junyang Wang et al. 2024. Mobile-agent-v2: Mobile device operation assistant with effective navigation via multi-agent collaboration. *arXiv preprint arXiv:2406.01014* (2024).

- [5] Hao Wen et al. 2024. AutoDroid-V2: Boosting SLM-based GUI Agents via Code Generation. *arXiv preprint arXiv:2412.18116* (2024).